

QChain

A post-quantum research blockchain

Project overview and roadmap

May 2026

<https://github.com/sil714/qchain>

What QChain is

QChain is a research blockchain that explores what happens when you combine three ideas that don't usually sit together in one system:

Post-quantum signatures. All transactions are signed with CRYSTALS-Dilithium — one of the NIST-standardized signature schemes designed to resist attacks from quantum computers. A future adversary with a useful quantum machine cannot forge QChain signatures the way they could forge ECDSA or RSA.

Quantum randomness from a real source. Leader election for block production draws randomness from the IBM Quantum Network's quantum random number generator — physical randomness from a quantum device, not a pseudo-random function seeded by some classical state.

Shielded payments via zero-knowledge proofs. Two privacy pools (a denomination-based mixer and a STARK-anonymized pool) let users hide transaction values and participants behind zk-STARK proofs, built on the Goldilocks-field AIR system from Microsoft's Winterfell library.

The result is a working chain: TCP peer-to-peer gossip, mempool, fork resolution, block production, multiple shielded-payment pools, and a live FastAPI dashboard. It runs end-to-end. You can mine blocks, deposit into the mixer, withdraw anonymously, and watch the whole thing happen in a browser.

What QChain is not. A production blockchain. A peer-reviewed cryptographic construction. A platform for actual money. The project is a **research demo** — built with the discipline of something more serious, but explicitly designed for exploration and review, not deployment.

The system at a glance

Cryptographic primitives	CRYSTALS-Dilithium (signatures), Rescue-Prime (Merkle hash), Goldilocks-field AIRs (zk-STARKs via Winterfell)
Consensus modes	Proof-of-work + Proof-of-stake (QRNG-seeded proposer)
Privacy mechanisms	M4 anonymity pool, denomination-based mixer pool, STARK-anonymized pool with partial spends
Network layer	TCP gossip, mempool, candidate-chain fork resolution, per-peer rate limiting
Wallet	Dilithium keypair, shielded-note tracking, optional argon2id + AES-256-GCM encryption at rest
Dashboard	FastAPI server with React UI, WebSocket event stream, bearer-token authentication
Persistence	Versioned JSON; chain and wallet survive restart
Codebase	Three layers — Rust AIR core (<code>qstark</code>), PyO3 bindings (<code>qstark_py</code>), Python chain + network + dashboard (<code>qchain</code>)

Project state at the time of writing:

Metric	Value
Total tests, all green	443
Maintained Markdown documents	37
Numbered threats in threat model	23
Threats actively defended	20
Self-found bugs/defects, all disclosed	5
Source code lines (approximate)	~15K

What’s distinctive

The cryptographic primitives in QChain are well-known. What’s unusual about this project, and what would matter to a thoughtful external reader, is the **engineering discipline applied to documenting and testing the system**. Four specific habits set it apart from a typical research-grade blockchain implementation:

Every claim has code and tests

Every defended attack in the threat model carries explicit references to the code that defends it and the tests that exercise the defense. Reading the threat model is reading a tour of the codebase. A claim without code or tests is not a claim — it’s marked [NOT DEFENDED] and listed alongside the defended ones, so a reader can see the full picture rather than the curated highlight reel.

A five-tier status taxonomy, not “defended/not defended”

Threats carry one of five status markers: [FORMAL] (defended under a standard cryptographic assumption), [FORMAL, MODULO] (defended under a standard assumption *and* a project-specific assumption like AIR correctness), [DEFENDED] (defended by application-layer mechanism with code and tests), [HEURISTIC] (defended by partial or best-effort mechanism), and [NOT DEFENDED] (explicitly out of scope). This taxonomy is unusual — most threat models in the literature use binary labels — but it accurately reflects the project’s actual epistemic state. A [FORMAL] claim and a [HEURISTIC] claim should not be presented as equally strong.

Self-disclosed findings

During the audit-readiness work, five bugs and defects were found and fixed by the project’s own review — all are documented. They include a coinbase-inflation flaw (severity: critical in production), a transparent-transaction replay bug found by Hypothesis property testing, an infinite-loop protocol bug surfaced when adding rate limiting, and two documentation defects (stale docstrings) found by cross-checking code against the docs claiming to describe it.

A project that finds and discloses its own bugs is sending a stronger signal than a project with nothing to disclose.

Differential testing of the cryptographic core

The zk-STARK trace is cross-checked, row by row, against an **independent Python re-execution** of the Rescue-Prime hash function. The Python implementation derives the same intermediate values from first principles, using constants extracted directly from the Winterfell crypto library so the two implementations share no code. This specifically targets the class of bug where the circuit is wrong but positive examples still work — the hardest failure mode for an AIR to detect, and the one that would most hurt an external auditor’s trust in the system.

The differential approach is not a proof of AIR correctness. It is evidence that several specific incorrect-AIR failure modes would have been caught. The distinction is made explicit in the documentation.

What’s been built

Major work falls into three eras:

Foundation (milestones M1–M8.5)

Working chain with transactions, blocks, signatures, proof-of-work mining, proof-of-stake leader election via QRNG, network layer, fork resolution, mempool, and the M4 anonymity pool. The first end-to-end zk-STARK transactions in the STARK-anonymized pool.

Cryptographic core (milestones M8.6–M8.11)

The AIR that powers shielded payments evolved through six milestones: nullifier binding, value conservation, the original depth-16 sparse Merkle tree, the depth-20 expansion, partial-spend change outputs, and the M8.10 admission-vs-replay consistency pattern that became a recurring discipline in the codebase. The mixer pool layer (M10, four phases) added denomination-based privacy and the M-timing defense against same-block linkability.

Audit-readiness arc (post-M10)

Eight passes specifically aimed at making the project legible and testable to an external reviewer:

1. **Audit-followup hardening** (five passes) — found coinbase inflation, added persistence versioning, hardened replay-path validation
2. **Mixer timing-attack defense** — historical-root anchoring with a `MIXER_WITHDRAWAL_DELAY` constant
3. **First audit package** — single-entry-point document with the ten major security claims cross-referenced to code
4. **Differential AIR validation** (two phases) — independent cross-checks of the STARK trace contents
5. **Property-based testing** with Hypothesis — found the transparent-tx replay bug
6. **Documentation cleanup** — threat model with five-tier status taxonomy
7. **Wallet encryption at rest** — argon2id + AES-256-GCM, closes T21
8. **Rate limiting + DoS hardening** — per-peer message limits, per-IP dashboard limits, max-block-size cap; closes T15 and T23, also fixed a same-height-fork protocol bug
9. **Wallet note lifecycle helpers** — reconciliation and pruning for orphaned shielded notes
10. **Dashboard authentication** — bearer-token auth on every API and WebSocket endpoint; closes T22 fully
11. **Publication writeup** — the 13-page external retrospective

What’s deliberately not done

This is the honest-scope list. An external reader should not have to discover any of these by reading the code.

- **No formal proof of AIR correctness.** Differential testing catches one class of bug but is not a proof. Several threats marked [FORMAL, MODULO] ride on AIR correctness as their non-formal assumption.
- **No Byzantine-fault-tolerant consensus.** The proof-of-stake mode uses a single QRNG-seeded proposer. A malicious-quorum scenario is out of scope; real PoS would need fault tolerance.
- **No peer authentication on the network layer.** Any node that can reach the TCP port and complete the handshake is treated as honest. Per-peer rate limiting bounds blast radius; it doesn’t prevent participation.
- **No TLS at any layer.** Both the network protocol and the dashboard bearer token travel in cleartext. Production deployments should put TLS-terminating reverse proxies in front of both.
- **No statistical timing-attack defense across many blocks.** The same-block defense (T13) only covers single-block correlation. An adversary with long-term observation could still infer deposit-to-withdrawal pairings via statistical analysis.
- **No on-load chain validation.** A corrupt-but-parseable persistence file would be accepted without protocol-level invariants being re-checked. Closing this is a known small follow-up.
- **No replay protection across networks.** Transactions on one chain instance can be replayed onto another. Not relevant for single-network use; would matter for multi-network deployment.
- **No off-chain note backup.** If a wallet file is lost, the shielded notes are not recoverable from the chain alone — by design, the chain holds only leaf commitments.
- **No peer-reviewed cryptographic analysis.** The audit-readiness arc is engineering discipline. It is preparation for peer review, not a substitute for it.

Roadmap

The roadmap is structured in three tiers: shipped work, planned work, and the **anti-roadmap** — things explicitly NOT being pursued and the reasons why.

Tier 1 — Recent next-up work

Of the six planned items in the audit-readiness arc’s roadmap, four have shipped, one is recommended (external engagement, not engineering work), and one remains open with low expected ROI. A bonus item — dashboard authentication — was added and shipped during the arc.

Item	Status
1.1 External audit engagement	Recommended; awaiting external engagement
1.2 Differential AIR Phase 3 (constants cross-reference)	Open, low expected ROI
1.3 Publication writeup	Shipped May 2026
1.4 Wallet key encryption at rest	Shipped May 2026
1.5 Rate limiting / DoS hardening (T15, T22, T23)	Shipped May 2026
1.6 Persistent wallet shielded-note tracking	Shipped May 2026
Dashboard authentication (bonus; closes T22 fully)	Shipped May 2026

The two remaining tier-1 items have specific characters worth noting:

- **1.1 (external audit)** is a calendar/budget item, not an engineering item. The project is materially ready for engagement. This is the single highest-leverage next step.
- **1.2 (Phase 3 constants cross-reference)** would verify the MDS and round constants of Rescue-Prime against a third independent implementation. Single-session scope; expected to pass cleanly because the constants are already validated against two implementations.

Tier 2 — Substantive future work

These are bigger pieces of work that would meaningfully expand the project’s capabilities or rigor:

2.1 Recursive STARKs / proof aggregation. Build a STARK that verifies other STARKs, so a block can carry one aggregate proof instead of N individual ones. Significant performance and scalability upgrade. Multi-month work.

2.2 Real Byzantine-fault-tolerant consensus. Replace the current PoW + QRNG-seeded-proposer with a true BFT protocol (Tendermint-style or similar) that withstands a malicious quorum fraction. Touches every layer of the system; large project.

2.3 Multi-asset support. Generalize the single-asset chain to track multiple asset IDs (tokens, NFTs, etc.). Moderate scope but ripples through transactions, wallet, and dashboard.

2.4 Formal verification of AIR polynomial constraints. Use a SAT/SMT solver (Z3, CVC5) or proof assistant (Lean, Coq) to verify the AIR’s transition constraints. The high-leverage long-term win for trustworthiness; needs a specialist.

Tier 3 — Anti-roadmap (things deliberately NOT being pursued)

A roadmap that only lists what to do isn't honest. Equally important is what the project is **declining to do**, and why.

Switching cryptographic frameworks (Winterfell → Plonky2 / RISC Zero). The existing work is built on Winterfell. Switching frameworks would discard several months of investment for marginal gains — and the gains would be in throughput, not in the dimensions where this project is trying to be useful.

Rewriting in another language. Language choices in this space are tribal. The work to port the codebase to Rust-everywhere or Go-everywhere or Move-or-Cairo-or-whatever would not change the underlying engineering claims, only consume effort that could go toward those claims.

Adding new consensus features incrementally. Consensus is one of those areas where partial work is worse than none. A half-finished BFT mode would actively mislead readers about the project's safety. Better to leave it as a single-validator demo with that fact explicitly documented.

Side-channel hardening of the prover. Listed as [NOT DEFENDED] in the audit package. Real production deployment would need this, but it's a different research project — the techniques and review discipline are different from anything else in QChain.

Marketing-driven scope expansion. No comment needed. The codebase's distinctive feature is the discipline of saying what's defended and what isn't. Stretching the scope to chase trends would destroy that distinctive feature.

How to engage

Different reader paths, by intent:

If you want to evaluate the project quickly

Start with `SUMMARY.pdf` (one page) for the headline facts, then read `PUBLICATION.pdf` (13 pages) for the audit-readiness retrospective. Together they take maybe 25 minutes and cover everything in this document at greater depth.

If you are a security auditor or cryptographer

The single highest-leverage external contribution is an independent review of the STARK circuits for shielded payments. Several threats marked `[FORMAL, MODULO]` ride on AIR correctness as their non-formal assumption; that's the gap an external specialist could close in a way no amount of internal work can match.

The relevant artifacts: `AUDIT-PACKAGE.md` (claims C1–C10), `THREAT-MODEL.md` (per-threat code references), and the `qstark/` Rust AIR implementation with its 110 tests.

If you are a researcher considering collaboration

The interesting collaboration directions are (in roughly decreasing order of leverage): formal verification of the AIR constraints, recursive STARK / proof aggregation, Byzantine-fault-tolerant consensus design. The first is a paper-shaped collaboration; the others are codebase-shaped.

The project's bias toward honest scope means it's open about what it isn't, which makes it a less defensive collaboration partner than projects that need to protect their claims.

If you are a grant program or funder

The project's distinctive feature — engineering discipline applied to documenting and testing — is the kind of work that's hard to fund because it doesn't produce new primitives or splashy demos. It produces audit-readiness. If your program has room for that mode of work, the project would be a fit.

The next concrete funding need is **paying an external auditor** for the review described above. The work-product would be a security audit report that, alongside `PUBLICATION.pdf`, makes the project ready for serious deployment consideration.

If you are a student or technically curious reader

`README.md` and `DOCS.md` are the entry points to the codebase. The project has 37 markdown documents that describe each milestone with honest scope notes — it's an unusual artifact for learning how a real cryptographic system gets built incrementally, including the parts that didn't work the first time.

Contact and references

Repository: <https://github.com/sil714/qchain>

Recommended reading order:

1. SUMMARY.md — one-page overview
2. This document — project presentation
3. PUBLICATION.md — 13-page audit-readiness retrospective
4. AUDIT-PACKAGE.md — claims C1–C10 for auditors
5. THREAT-MODEL.md — 23-threat status table
6. DOCS.md — reading-order guide for the rest

Total project documentation: 37 Markdown files in the repository root, organized by milestone and by purpose. All cross- references between documents are kept current by an automated check in the engineering discipline of the project.

*QChain is a research demo. It is not production software.
It is built with the discipline of something more serious, and made
deliberately legible to someone considering external engagement.*